629_Final_Project

Guanyi Wu

2026-04-24

Reproducibility Instructions

1. Load packages

```
library(data.table)
library(survival)
library(randomForestSRC)
```

```
## Warning: package 'randomForestSRC' was built under R version 4.4.3
```

```
##
##  randomForestSRC 3.5.1
##
##  Type rfsrc.news() to see new features, changes, and bug fixes.
##
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:data.table':
##
##     between, first, last
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(tidyr)
library(stringr)
library(utils)
```

2. Download following dataset

```
ClinOutcomes <- read.delim(
  "https://media.githubusercontent.com/media/cBioPortal/datahub/refs/heads/master/public/msk_chord_2024/
  comment.char = "#",
  stringsAsFactors = TRUE
)
```

```r
DiagnosisFull <- read.delim2(
  "https://media.githubusercontent.com/media/cBioPortal/datahub/refs/heads/master/public/msk_chord_2024/
  comment.char = "#"
)

Samples <- read.delim(
  "https://media.githubusercontent.com/media/cBioPortal/datahub/refs/heads/master/public/msk_chord_2024/
  comment.char = "#"
)

Treatment <- read.delim(
  "https://media.githubusercontent.com/media/cBioPortal/datahub/refs/heads/master/public/msk_chord_2024/
  comment.char = "#"
)

CancerPresence <- read.delim(
  "https://media.githubusercontent.com/media/cBioPortal/datahub/refs/heads/master/public/msk_chord_2024/
  comment.char = "#"
)

TumorSites <- read.delim(
  "https://media.githubusercontent.com/media/cBioPortal/datahub/refs/heads/master/public/msk_chord_2024/
  comment.char = "#"
)

seg <- fread(
  "https://media.githubusercontent.com/media/cBioPortal/datahub/refs/heads/master/public/msk_chord_2024/
)

muta <- fread(
  "https://media.githubusercontent.com/media/cBioPortal/datahub/refs/heads/master/public/msk_chord_2024/
)
```

3. Precomputed Results

For models with long runtimes, I will directly import the results into this document. You can find corresponding files in the Github repository.

Data Processing

```r
# 1. mutation data -> patient-level binary indicators
genes <- c("TP53", "KRAS", "EGFR")

mut_patient <- muta %>%
  mutate(PATIENT_ID = str_extract(Tumor_Sample_Barcode, "P-[0-9]+")) %>%
  filter(
    Hugo_Symbol %in% genes,
    Mutation_Status == "SOMATIC"
  ) %>%
  group_by(PATIENT_ID, Hugo_Symbol) %>%
  summarise(mutated = 1, .groups = "drop") %>%
  pivot_wider(
```

```r
    names_from = Hugo_Symbol,
    values_from = mutated,
    values_fill = 0
  )

# 2. clinical outcome data
clin <- ClinOutcomes %>%
  dplyr::select(
    PATIENT_ID,
    OS_MONTHS,
    OS_STATUS,
    CURRENT_AGE_DEID,
    GENDER
  )

# 3. diagnosis data
diag <- DiagnosisFull %>%
  dplyr::select(
    PATIENT_ID,
    START_DATE,
    STAGE_CDM_DERIVED
  ) %>%
  filter(!is.na(PATIENT_ID)) %>%
  group_by(PATIENT_ID) %>%
  slice_min(order_by = abs(START_DATE), n = 1, with_ties = FALSE) %>%
  ungroup()

# 4. cancer type data
cancer <- Samples %>%
  dplyr::select(PATIENT_ID, CANCER_TYPE) %>%
  filter(!is.na(PATIENT_ID)) %>%
  distinct()

# 5. merge all data
data_model <- clin %>%
  left_join(diag, by = "PATIENT_ID") %>%
  left_join(cancer, by = "PATIENT_ID") %>%
  left_join(mut_patient, by = "PATIENT_ID")

# replace NA in mutation indicators with 0
data_model <- data_model %>%
  mutate(across(all_of(genes), ~ ifelse(is.na(.), 0, .)))

# 6. format converting
data_model <- data_model %>%
  filter(!is.na(OS_MONTHS), !is.na(OS_STATUS)) %>%
  mutate(
    GENDER = factor(GENDER),
    STAGE_CDM_DERIVED = factor(STAGE_CDM_DERIVED),
    CANCER_TYPE = factor(CANCER_TYPE)
  )

## convert OS_STATUS to 0/1
```

```r
data_model <- data_model %>%
  mutate(OS_STATUS = ifelse(grepl("^1", as.character(OS_STATUS)), 1, 0))

data_model <- data_model %>%
  drop_na(
    OS_MONTHS,
    OS_STATUS,
    CURRENT_AGE_DEID,
    GENDER,
    STAGE_CDM_DERIVED,
    CANCER_TYPE
  )

data_model$GENDER <- droplevels(data_model$GENDER)
```

IPCW Brier Score for survival data

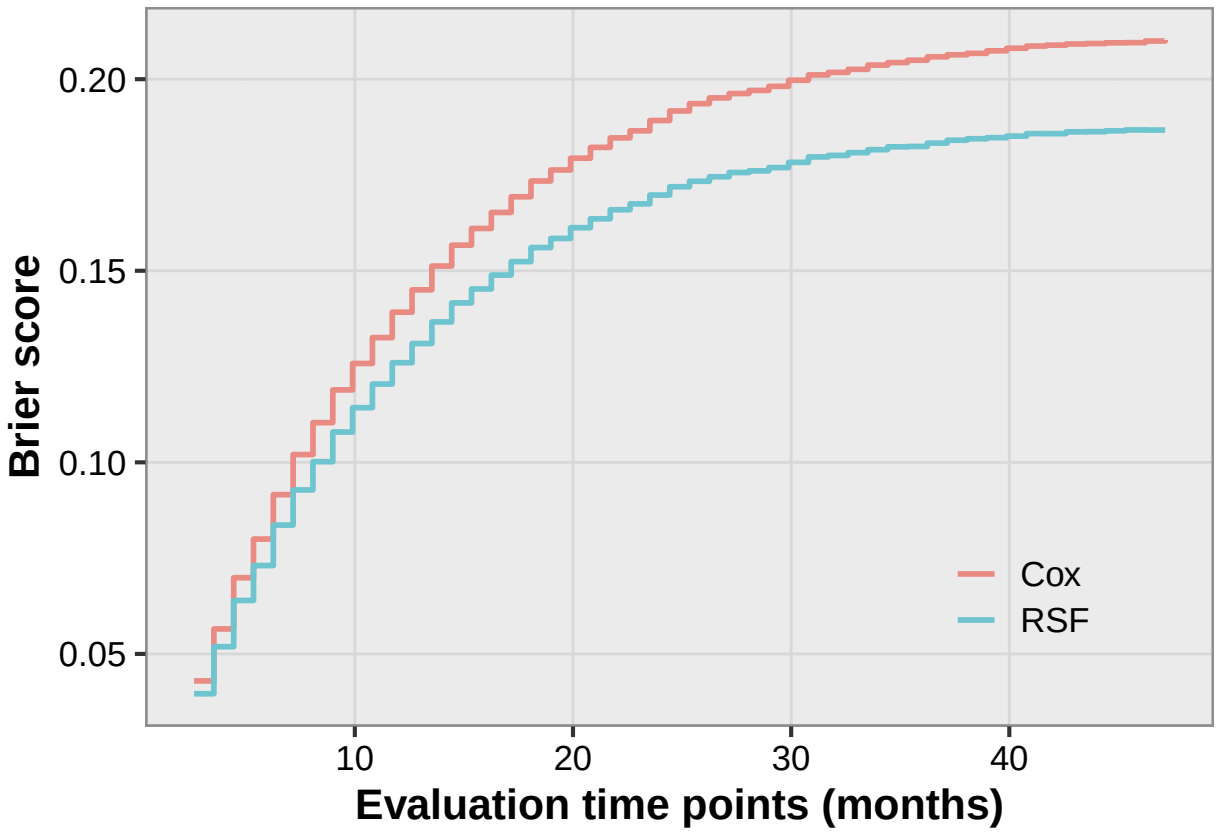
```r
# 1. choose evaluation time points
event_times <- data_model$OS_MONTHS[data_model$OS_STATUS == 1]

time_grid <- seq(
  quantile(event_times, 0.1),
  quantile(event_times, 0.9),
  length.out = 50
)

# 2. estimate censoring distribution G(t) by KM
km_censor <- survfit(Surv(OS_MONTHS, 1 - OS_STATUS) ~ 1, data = data_model)

## step function for G(t) = P(C > t)
get_Ghat <- stepfun(
  km_censor$time,
  c(1, km_censor$surv)
)

## left limit at t
get_Ghat_left <- function(t, eps = 1e-8) {
  pmax(get_Ghat(t - eps), 1e-6)
}

## helper: G(t)
get_Ghat_right <- function(t) {
  pmax(get_Ghat(t), 1e-6)
}

# # 3. predicted survival probabilities from Cox model
# cox_model <- coxph(
#   Surv(OS_MONTHS, OS_STATUS) ~
#     CURRENT_AGE_DEID +
#     GENDER +
#     STAGE_CDM_DERIVED +
#     CANCER_TYPE +
```

```r
#     TP53 + KRAS + EGFR,
#   data = data_model
# )
# sf <- survfit(cox_model, newdata = data_model)
# cox_surv_summary <- summary(sf, times = time_grid)
#
# n <- nrow(data_model)
# nt <- length(time_grid)
#
# ## reshape Cox predicted survival into n x nt matrix
# cox_surv_mat <- matrix(
#   cox_surv_summary$surv,
#   nrow = nt,
#   ncol = n
# )
#
# cox_surv_mat <- t(cox_surv_mat)
# write.csv(cox_surv_mat, "cox_surv_mat.csv", row.names = FALSE)

cox_surv_mat <- read.csv("cox_surv_mat.csv")
## rows = subjects, cols = time points
#colnames(cox_surv_mat) <- paste0("t_", seq_len(nt))

# 4. predicted survival probabilities from RSF model
# rsf_model <- rfsrc(
#   Surv(OS_MONTHS, OS_STATUS) ~
#     CURRENT_AGE_DEID +
#     GENDER +
#     STAGE_CDM_DERIVED +
#     CANCER_TYPE +
#     TP53 + KRAS + EGFR,
#   data = data_model,
#   ntree = 600,
#   nodesize = 15,
#   nsplit = 10
# )
# saveRDS(rsf_model, "rsf_model.rds")
#
# rsf_pred <- predict(rsf_model, newdata = data_model)
#
# saveRDS(rsf_pred, "rsf_pred.rds")
# rsf_model <- readRDS("rsf_model.rds")
# rsf_pred <- readRDS("rsf_pred.rds")
#
# rsf_surv_all  <- rsf_pred$survival
# rsf_time_all  <- rsf_pred$time.interest
#
# ## interpolate RSF survival at chosen time_grid
# rsf_surv_mat <- t(apply(rsf_surv_all, 1, function(s) {
#   approx(
#     x = rsf_time_all,
#     y = s,
#     xout = time_grid,
```

```r
#     method = "linear",
#     rule = 2
#   )$y
# }))
# write.csv(rsf_surv_mat, "rsf_surv_mat.csv", row.names = FALSE)
rsf_surv_mat <- read.csv("rsf_surv_mat.csv")

#colnames(rsf_surv_mat) <- paste0("t_", seq_len(nt))

# 5. function to compute IPCW Brier score at one time t
ipcw_brier_one_time <- function(time, status, surv_pred_t, t_eval) {

  y_t <- as.numeric(time > t_eval)

  w <- ifelse(
    time <= t_eval & status == 1,
    1 / get_Ghat_left(time),
    ifelse(time > t_eval,
           1 / get_Ghat_right(t_eval),
           0)
  )

  mean(w * (y_t - surv_pred_t)^2)
}

# 6. compute Brier score curve over time grid
brier_cox <- sapply(seq_along(time_grid), function(j) {
  ipcw_brier_one_time(
    time = data_model$OS_MONTHS,
    status = data_model$OS_STATUS,
    surv_pred_t = cox_surv_mat[, j],
    t_eval = time_grid[j]
  )
})

brier_rsf <- sapply(seq_along(time_grid), function(j) {
  ipcw_brier_one_time(
    time = data_model$OS_MONTHS,
    status = data_model$OS_STATUS,
    surv_pred_t = rsf_surv_mat[, j],
    t_eval = time_grid[j]
  )
})

# 7. integrated Brier score (IBS)
trapz <- function(x, y) {
  sum(diff(x) * (head(y, -1) + tail(y, -1)) / 2)
}

ibs_cox <- trapz(time_grid, brier_cox) / (max(time_grid) - min(time_grid))
ibs_rsf <- trapz(time_grid, brier_rsf) / (max(time_grid) - min(time_grid))

# 8. combine results into a data frame
```

```r
brier_results <- data.frame(
  time = time_grid,
  Brier_Cox = brier_cox,
  Brier_RSF = brier_rsf
)

# 9. plot Brier score curves
plot_dat <- data.frame(
  Time = rep(brier_results$time, 2),
  Brier = c(brier_results$Brier_Cox, brier_results$Brier_RSF),
  Model = factor(
    rep(c("Cox", "RSF"), each = nrow(brier_results)),
    levels = c("Cox", "RSF")
  )
)

ggplot(plot_dat, aes(x = Time, y = Brier, color = Model)) +
  geom_step(linewidth = 1) +
  scale_color_manual(
    values = c("Cox" = "#E98B83", "RSF" = "#6EC5CF")
  ) +
  labs(
    x = "Evaluation time points (months)",
    y = "Brier score",
    color = NULL
  ) +
  theme_grey(base_size = 16) +
  theme(
    panel.border = element_rect(color = "#8F8F8F", fill = NA, linewidth = 0.8),
    panel.grid.major = element_line(color = "#D8D8D8", linewidth = 0.5),
    panel.grid.minor = element_line(color = "#ECECEC", linewidth = 0.3),
    legend.position = c(0.82, 0.18),
    legend.background = element_blank(),
    legend.key = element_blank(),
    axis.title = element_text(face = "bold"),
    axis.text = element_text(color = "black")
  )
```

```
## Warning: A numeric `legend.position` argument in `theme()` was deprecated in ggplot2
## 3.5.0.
## i Please use the `legend.position.inside` argument of `theme()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Predictive Gain based on Integrated Brier Score

```
# 1. variables in the full multimodal RSF model
vars_full <- c(
  "CURRENT_AGE_DEID",
  "GENDER",
  "STAGE_CDM_DERIVED",
  "CANCER_TYPE",
  "TP53",
  "KRAS",
  "EGFR"
)

# 2. evaluation time points
event_times <- data_model$OS_MONTHS[data_model$OS_STATUS == 1]

time_grid <- seq(
  quantile(event_times, 0.1),
  quantile(event_times, 0.9),
  length.out = 20
)

# 3. IBS function using OOB survival matrix
# get_ipcw_brier_rsf_oob <- function(surv_mat, pred_times, data, eval_times) {
#
```

```
#   time <- data$OS_MONTHS
#   status <- data$OS_STATUS
#
#   ## Kaplan-Meier for censoring distribution: G(t) = P(C > t)
#   km_censor <- survfit(Surv(time, 1 - status) ~ 1, data = data)
#
#   G_fun <- stepfun(
#     km_censor$time,
#     c(1, km_censor$surv)
#   )
#
#   G_left <- function(t, eps = 1e-8) {
#     pmax(G_fun(t - eps), 1e-6)
#   }
#
#   G_right <- function(t) {
#     pmax(G_fun(t), 1e-6)
#   }
#
#   ## interpolate OOB survival probabilities to eval_times
#   surv_eval <- t(apply(surv_mat, 1, function(s) {
#     approx(
#       x = pred_times,
#       y = s,
#       xout = eval_times,
#       method = "linear",
#       rule = 2
#     )$y
#   }))
#
#   ## time-dependent IPCW Brier score
#   brier <- sapply(seq_along(eval_times), function(j) {
#
#     t0 <- eval_times[j]
#
#     y_t <- as.numeric(time > t0)
#
#     w <- ifelse(
#       time <= t0 & status == 1,
#       1 / G_left(time),
#       ifelse(
#         time > t0,
#         1 / G_right(t0),
#         0
#       )
#     )
#
#     mean(w * (y_t - surv_eval[, j])^2)
#   })
#
#   ## integrated Brier score by trapezoidal rule
#   ibs <- sum(diff(eval_times) * (head(brier, -1) + tail(brier, -1)) / 2) /
#     (max(eval_times) - min(eval_times))
```

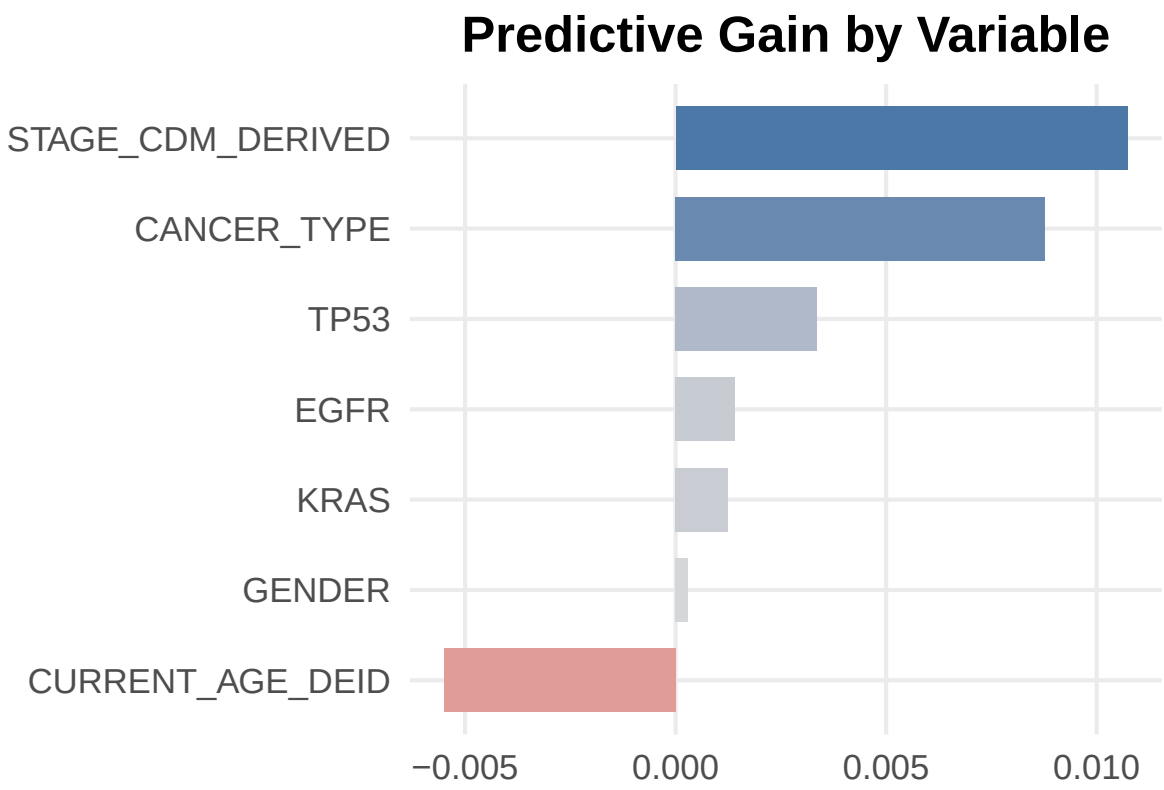
```
#
#   list(
#     brier = brier,
#     ibs = ibs
#   )
# }

# 4. full model IBS using OOB survival prediction
# full_perf <- get_ipcw_brier_rsf_oob(
#   surv_mat = rsf_model$survival.oob,
#   pred_times = rsf_model$time.interest,
#   data = data_model,
#   eval_times = time_grid
# )
#
# ibs_full <- full_perf$ibs
#
# cat("Full model IBS =", round(ibs_full, 4), "\n")

# 5. leave-one-variable-out predictive gain
# predictive_gain_results <- vector("list", length(vars_full))
#
# for (i in seq_along(vars_full)) {
#
#   v <- vars_full[i]
#   cat("Running reduced model without:", v, "\n")
#
#   vars_reduced <- setdiff(vars_full, v)
#
#   reduced_formula <- as.formula(
#     paste(
#       "Surv(OS_MONTHS, OS_STATUS) ~",
#       paste(vars_reduced, collapse = " + ")
#     )
#   )
#
#   rsf_model_reduced <- rfsrc(
#     reduced_formula,
#     data = data_model,
#     ntree = 200,
#     nodesize = 15,
#     nsplit = 10
#   )
#
#   reduced_perf <- get_ipcw_brier_rsf_oob(
#     surv_mat = rsf_model_reduced$survival.oob,
#     pred_times = rsf_model_reduced$time.interest,
#     data = data_model,
#     eval_times = time_grid
#   )
#
#   ibs_reduced <- reduced_perf$ibs
#
```

```r
#   predictive_gain <- ibs_reduced - ibs_full
#
#   predictive_gain_results[[i]] <- data.frame(
#     Variable = v,
#     IBS_full = ibs_full,
#     IBS_reduced = ibs_reduced,
#     Predictive_Gain = predictive_gain
#   )
#
#   rm(rsf_model_reduced, reduced_perf)
#   gc()
# }
#
# predictive_gain_results <- bind_rows(predictive_gain_results) %>%
#   arrange(desc(Predictive_Gain))
#
# saveRDS(predictive_gain_results, "predictive_gain_results.rds")
predictive_gain_results <- readRDS("predictive_gain_results.rds")
print(predictive_gain_results)
```

```
##            Variable  IBS_full IBS_reduced Predictive_Gain
## 1 STAGE_CDM_DERIVED 0.1752324   0.1859664     0.010733993
## 2       CANCER_TYPE 0.1752324   0.1840010     0.008768565
## 3              TP53 0.1752324   0.1785824     0.003349962
## 4              EGFR 0.1752324   0.1766455     0.001413061
## 5              KRAS 0.1752324   0.1764853     0.001252869
## 6            GENDER 0.1752324   0.1755227     0.000290279
## 7  CURRENT_AGE_DEID 0.1752324   0.1697340    -0.005498432
```

```r
# 6. plot predictive gain
ggplot(
  predictive_gain_results,
  aes(
    x = reorder(Variable, Predictive_Gain),
    y = Predictive_Gain,
    fill = Predictive_Gain
  )
) +
  geom_col(width = 0.7) +
  coord_flip() +
  scale_fill_gradient2(
    low = "#D95F5F",
    mid = "#D9D9D9",
    high = "#4C78A8",
    midpoint = 0
  ) +
  labs(
    title = "Predictive Gain by Variable",
    x = "",
    y = ""
  ) +
  theme_minimal(base_size = 16) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
```

```
    legend.position = "none",
    panel.grid.minor = element_blank()
  )
```



Uncertainty Quantification

```
set.seed(123)

# 1. keep only required columns
data_conformal <- data_model %>%
  dplyr::select(
    OS_MONTHS,
    OS_STATUS,
    all_of(vars_full)
  ) %>%
  drop_na()

data_conformal$GENDER <- droplevels(factor(data_conformal$GENDER))
data_conformal$STAGE_CDM_DERIVED <- droplevels(factor(data_conformal$STAGE_CDM_DERIVED))
data_conformal$CANCER_TYPE <- droplevels(factor(data_conformal$CANCER_TYPE))

# 2. split data into training set D1 and calibration set D2
n <- nrow(data_conformal)
train_idx <- sample(seq_len(n), size = floor(0.7 * n))
train_data <- data_conformal[train_idx, ]
```

```r
cal_data   <- data_conformal[-train_idx, ]

cat("Training set size:", nrow(train_data), "\n")
```

```
## Training set size: 17521
```

```r
cat("Calibration set size:", nrow(cal_data), "\n")
```

```
## Calibration set size: 7509
```

```r
# 3. fit RSF on training set
# rsf_formula <- as.formula(
#   paste("Surv(OS_MONTHS, OS_STATUS) ~", paste(vars_full, collapse = " + "))
# )
#
# rsf_model_conf <- rfsrc(
#   formula = rsf_formula,
#   data = train_data,
#   ntree = 700,
#   nodesize = 15,
#   nsplit = 10
# )

# 4. function to extract predicted median survival time
get_median_surv_rsf <- function(pred) {
  surv <- pred$survival
  time <- pred$time.interest

  med <- apply(surv, 1, function(s) {
    idx <- which(s <= 0.7)[1]
    if (is.na(idx)) return(NA_real_)
    time[idx]
  })

  med
}

# 5. predict on calibration set
# pred_cal <- predict(rsf_model_conf, newdata = cal_data)
# cal_data$pred_median <- get_median_surv_rsf(pred_cal)
#
# write.csv(cal_data, "cal_data.csv", row.names = FALSE)
cal_data <- read.csv("cal_data.csv")
# 6. compute residuals on calibration set
cal_obs <- cal_data %>% filter(OS_STATUS == 1, !is.na(pred_median)) %>% mutate( abs_residual = abs(OS_M

# 7. choose confidence level and compute conformal quantile
alpha <- 0.05   ## 95% prediction interval

n_cal <- nrow(cal_obs)

qhat <- quantile(
  cal_obs$abs_residual,
  probs = ceiling((n_cal + 1) * (1 - alpha)) / n_cal,
  type = 1,
```

```r
  na.rm = TRUE
)

# 8. point prediction + prediction interval for all subjects
# pred_all <- predict(rsf_model_conf, newdata = data_conformal)
# data_conformal$pred_median <- get_median_surv_rsf(pred_all)
# data_conformal <- data_conformal %>% mutate( PI_lower = pmax(pred_median - qhat, 0), PI_upper = pred_
#write.csv(data_conformal, "data_conformal.csv", row.names = FALSE)
data_conformal <- read.csv("data_conformal.csv")
# 9. empirical coverage on observed deaths only
coverage_data <- data_conformal %>%
  filter(OS_STATUS == 1, !is.na(pred_median)) %>%
  mutate(
    covered = (OS_MONTHS >= PI_lower & OS_MONTHS <= PI_upper)
  )

empirical_coverage <- mean(coverage_data$covered)
avg_width <- mean(coverage_data$PI_upper - coverage_data$PI_lower)

cat("Empirical coverage on observed deaths =", round(empirical_coverage, 3), "\n")
```

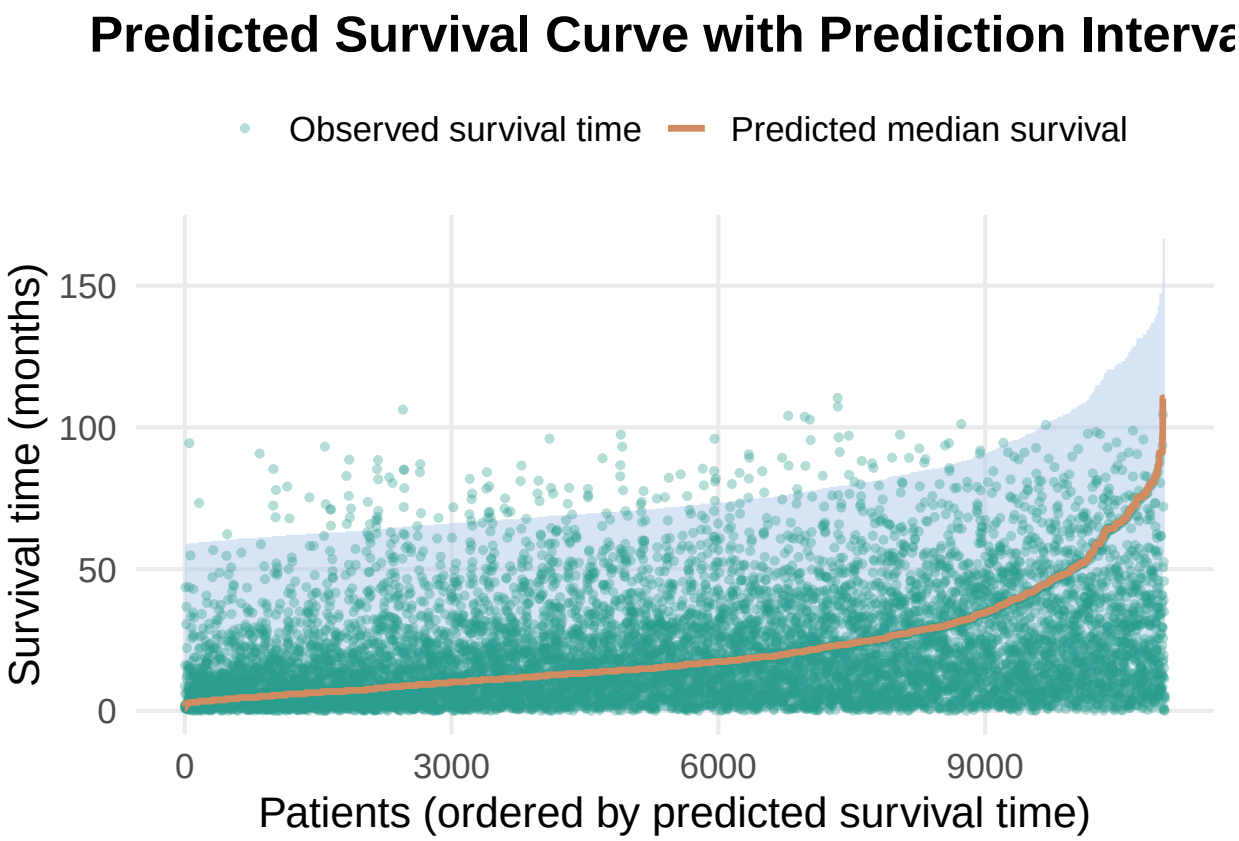
```
## Empirical coverage on observed deaths = 0.964
```

```r
cat("Average interval width =", round(avg_width, 3), "\n")
```

```
## Average interval width = 77.145
```

```r
# 10. plot prediction intervals
plot_curve_data2 <- data_conformal %>%
  filter(OS_STATUS == 1, !is.na(pred_median), !is.na(PI_lower), !is.na(PI_upper)) %>%
  arrange(pred_median) %>%
  mutate(ID = row_number())

ggplot(plot_curve_data2, aes(x = ID)) +


  geom_ribbon(aes(ymin = PI_lower, ymax = PI_upper),
              fill = "#AFCBEA", alpha = 0.5) +


  geom_point(aes(y = OS_MONTHS, color = "Observed survival time"),
             size = 1.1,
             alpha = 0.35) +

  geom_line(aes(y = pred_median, color = "Predicted median survival"),
            linewidth = 1.2) +

  scale_color_manual(
    values = c(
      "Predicted median survival" = "#D08C60",
      "Observed survival time" = "#2A9D8F"
    )
  ) +

  labs(
```

```
  title = "Predicted Survival Curve with Prediction Interval",
  x = "Patients (ordered by predicted survival time)",
  y = "Survival time (months)",
  color = NULL
) +

theme_minimal(base_size = 16) +
theme(
  plot.title = element_text(face = "bold", hjust = 0.5),
  legend.position = "top",
  panel.grid.minor = element_blank()
)
```



Website Link

https://guanyiwu0823.github.io/msk-chord-survival/